

Books Plot Search Service with MongoDB Atlas



Estimated time needed: **30** minutes

Objectives

After completing this lab, you will be able to:

- work with csv file in a Node.js application
- Generate vector embeddings for book plot data using models
- Connect to the MongoDB Atlas database and create a table with vector embeddings
- Connect to the MongoDB Atlas database and create a table with vector embeddings

About Skills Network Cloud IDE

Skills Network Cloud IDE (based on Theia and Docker) is an open-source Integrated Development Environment (IDE) that provides an environment for hands-on labs in courses and projects.

Important notice

Please be aware that sessions for this lab environment are not persistent. Every time you connect to this lab, a new environment is created for you. Any data you may have saved in the earlier session will be lost. Plan to complete these labs in a single session to avoid losing your data.

Running the lab

Currently, the Skills Network Cloud IDE, which utilizes Theia and Docker technologies, does not support this lab. You have two options to participate in this **optional** lab.

1. Use MongoDB Atlas on MongoDB Cloud as a fully managed service by following [steps in this lab](#).
2. Optionally, you can install MongoDB Atlas locally on your computer using Docker by following the [steps in this lab](#).

This lab assumes that you have picked option 1 above.

Prerequisites

MongoDB Atlas on the Cloud

Ensure you have created an account on MongoDB Cloud and provisioned an Atlas database by following [steps in this lab](#).

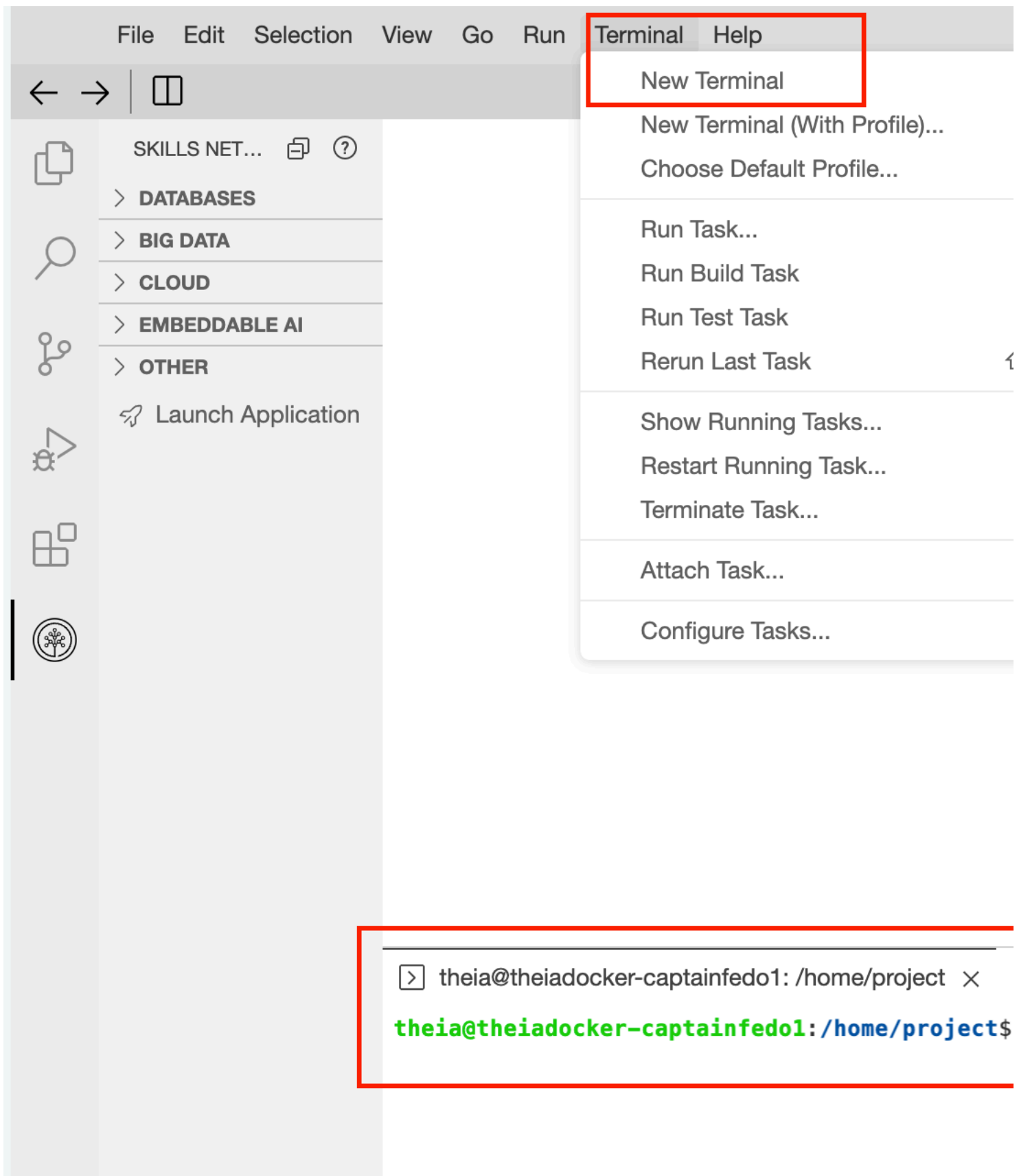
Create directories

In this lab, you will work with MongoDB Atlas, **not** the regular MongoDB database. The Atlas version simplifies the vector storage, indexing, and searching processes, and you will use all the features in this lab. You will search for similar news items using the `cosine` function. Here is a list of all the similarity functions available, along with their descriptions:

- `cosine`: Measures the cosine of the angle between two vectors. A higher cosine value indicates more remarkable similarity.
- `euclidean`: Measures the straight-line distance between two points (vectors) in a multidimensional space. A lower distance indicates higher similarity.
- `dotproduct` calculates the sum of the product of corresponding components between two vectors. Higher dot product values indicate greater alignment.

Note: [You can learn more about it in the MongoDB Atlas vector search documentation.](#)

Open a terminal in the IDE by using the `Terminal` -> `New Terminal` menu. You should see the terminal appear at the bottom of the screen.



```
mkdir mongodb-book-vector && cd mongodb-book-vector
```

If you are starting this lab immediately after completing an initial lab, please delete all folders before continuing.

Let's also save the full path of the folder in an environment variable that we can refer to later.

```
export VECTOR_HOME=$(pwd)
```

You can use the echo command to see what is inside this variable.

```
echo $VECTOR_HOME
```

The output looks as follows. It will look different on your computer.

```
/home/project/mongodb-book-vector
```

Let's create a data sub-directory using this variable.

```
mkdir $VECTOR_HOME/data
```

Initial Data

The system stores the book plot data in the `bookPlots.csv` file. You worked with JSON format in the previous lab, but CSV is another common format for storing data. Each book plot item has a unique `id`, name of the book (`book_name`), summaries, and category to which the book belongs. It will look like the following in the database:

```
id,book_name,summaries,categories
0,The Highly Sensitive Person,"is a self-assessment guide and how-to-live template for people who feel, relate, process, and notice more deeply than
1,Why Has Nobody Told Me This Before?," is a collection of a clinical psychologists best practical advice to combat anxiety and depression and improv
```

Data attribution

"Books Summary Dataset" by Arpan Srivastava is licensed under [CC0: Public Domain](#)

You can get the complete file using the following `wget` or `curl` command or simply opening this [URL](#) in your browser and saving the file locally.

WGET

```
wget -O $VECTOR_HOME/data/books-summary.csv https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/_xKx951b9LB03QU8b11M4A/books-summary.c
```

CURL

```
curl -o $VECTOR_HOME/data/books-summary.csv https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/_xKx951b9LB03QU8b11M4A/books-summary.c
```

Task 1: Create Node.js project and install dependencies

Let's create the node package and install all the dependencies.

1. Ensure you are still in the `$VECTOR_HOME` directory and run `npm init -y` command.

```
npm init -y
```

The output should look like:

```
Wrote to /home/project/mongodb-book-vector/package.json:
{
  "name": "mongodb-book-vector",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC"
}
```

2. Install the required packages including, `@tensorflow-models/universal-sentence-encoder`, `@tensorflow/tfjs-node`, `dotenv`, `mongodb`, `express` and `csv-parser`.
 - `@tensorflow-models/universal-sentence-encoder`: The universal sentence encoder model
 - `@tensorflow/tfjs-node`: JavaScript version of TensorFlow

- dotenv: Loads environment variables from a.env file into process.env
- mongodb: Library to work with mongoDB
- express: Web framework for Node.js.

```
npm install @tensorflow-models/universal-sentence-encoder @tensorflow/tfjs-node dotenv express mongodb csv-parser
```

The output is long but should end with libraries installed:

```
added 197 packages, and audited 198 packages in 42s
21 packages are looking for funding
  run `npm fund` for details
2 moderate severity vulnerabilities
To address all issues, run:
  npm audit fix
Run `npm audit` for details.
```

You can safely ignore the warnings for now and verify that all packages were installed using the `npm list` command:

```
npm list
```

The output should show the packages you just installed.

```
├─ @tensorflow-models/universal-sentence-encoder@1.3.3
├─ @tensorflow/tfjs-node@4.19.0
├─ csv-parser@3.0.0
├─ dotenv@16.4.5
├─ express@4.19.2
└─ mongodb@6.6.1
```

Task 2: Set up configuration

1. Create a file called `.env` in the `$VECTOR_HOME` directory.

```
touch .env
```

2. Populate the file with the following information:

```
MONGO_HOST=  
MONGO_PORT=27017  
MONGO_USER=  
MONGO_PASS=  
MONGO_DB=booksdb  
MONGO_COLLECTION=books_vector
```

Refer to the section Task 3: Load the vectors into MongoDB Atlas in the lab [News Finder Service with MongoDB Atlas](#) on how to obtain the host name, username and password from MongoDB Cloud.

Task 3: Create common util functions

We will reuse the util.js file from the previous lab [News Finder Service with MongoDB Atlas](#).

Create util.js file with common reusable code

Create a new file name util.js.

```
touch util.js
```

Here is the complete file if you are stuck

► [Click here for the file](#)

The previous lab goes through this code in detail.

Task 4: Load the vectors into MongoDB Atlas

File to upload vectorized data into the collection

Create a new file named loader.js.

```
touch loader.js
```

This file will contain the method that loads the initial book plots to the MongoDB database.

```
const fs = require('fs');  
const csv = require('csv-parser');  
const { vectorizeText, connectToMongoDB, closeMongoDBConnection, MONGO_COLLECTION } = require('./util');  
async function loadVectorizedData() {  
  const db = await connectToMongoDB();  
  const collection = db.collection(MONGO_COLLECTION);  
  try {  
    const results = [];  
    fs.createReadStream('data/books-summary.csv')  
      .pipe(csv())  
      .on('data', (data) => results.push(data))
```

```

.on('end', async () => {
  for (const book of results) {
    const text = book.summaries;
    const description_vector = await vectorizeText(text);
    if (description_vector && description_vector.length > 0) {
      await collection.insertOne({
        id: book.id,
        book_name: book.book_name,
        summaries: book.summaries,
        categories: book.categories,
        description_vector: description_vector
      });
      console.log('Vectorized data stored in MongoDB Atlas for:', book.book_name);
    } else {
      console.log('No vectors were generated for:', book.book_name);
    }
  }
  await closeMongoDBConnection();
});
} catch (error) {
  console.error('Error during data loading:', error);
  await closeMongoDBConnection();
}
}
loadVectorizedData();

```

The only change from the last lab is that we are loading the data from a CSV file instead of a JSON file.

```

fs.createReadStream('data/books-summary.csv')
  .pipe(csv())

```

Run the code

1. Install the packages
Install all the packages using the npm install command.

```
npm install
```

2. Run the file
Run the file using the node command.

```
node loader.js
```

You should see the new articles load one by one:

```

Platform node has already been set. Overwriting the platform with node.
mongodb://root:root@localhost:27017?directConnection=true

```

```
Connected to MongoDB Atlas
Vectorized data stored in MongoDB Atlas for: The Highly Sensitive Person
Vectorized data stored in MongoDB Atlas for: Why Has Nobody Told Me This Before?
Vectorized data stored in MongoDB Atlas for: The Midnight Library
Vectorized data stored in MongoDB Atlas for: Brave New World
Vectorized data stored in MongoDB Atlas for: 1984
Vectorized data stored in MongoDB Atlas for: Stolen Focus
Vectorized data stored in MongoDB Atlas for: The Life-Changing Science of Detecting Bullshit
Vectorized data stored in MongoDB Atlas for: Dopamine Nation
Vectorized data stored in MongoDB Atlas for: The Art of Statistics
Vectorized data stored in MongoDB Atlas for: No Self No Problem
Vectorized data stored in MongoDB Atlas for: The Highly Sensitive Person
Vectorized data stored in MongoDB Atlas for: Why Has Nobody Told Me This Before?
Vectorized data stored in MongoDB Atlas for: The Midnight Library
Vectorized data stored in MongoDB Atlas for: Brave New World
Vectorized data stored in MongoDB Atlas for: 1984
Vectorized data stored in MongoDB Atlas for: Stolen Focus
Vectorized data stored in MongoDB Atlas for: The Life-Changing Science of Detecting Bullshit
Vectorized data stored in MongoDB Atlas for: Dopamine Nation
Vectorized data stored in MongoDB Atlas for: The Art of Statistics
Vectorized data stored in MongoDB Atlas for: No Self No Problem
Vectorized data stored in MongoDB Atlas for: The Code Breaker
Vectorized data stored in MongoDB Atlas for: Super Human
MongoDB connection closed
```

If you get no error, it means the articles were vectorized and loaded into the collection.

Task 5: Create an index based on vector data

Log into the database using Mongosh

Having stored the short descriptions of book plots in the database as 512-dimensional vectors, creating an index on this vector attribute is essential. This step enables us to leverage Atlas's capabilities for conducting vector searches, allowing us to find items with similar content efficiently.

You have already done this in the [News Finder Service with MongoDB Atlas](#) lab under Task 4: Create index based on the vector data. Follow the steps in that lab to create an index for this data.

- pick the booksdb database
- pick the books_vector collection
- pick the description_vector as the field to index

Your screen should look like this:

Create a Vector Search Index



JSON Editor

[View Atlas Vector Search Docs](#)

Search Index Type: **vectorSearch**

Give your vector search index a name for easy reference, and select the database and collection you want to draw data from. You can also create, edit, and manage search indexes using the [Atlas API](#).

Database and Collection

- ▼ booksdb
 - books_vector
- > myvectordb
- > sample_mflix
- > sample-test

Index Name

```
1 {
2   "fields": [
3     {
4       "type": "vector",
5       "path": "description_vector",
6       "numDimensions": 512,
7       "similarity": "cosine"
8     }
9   ]
10 }
```

BackCancelNext

Once the index completes, you should see a status of active:

booksdb.books_vector

Indexes Used: 2 of 3.

Name	Index Type	Index Fields	Status ⓘ	Size	Documents	Actions
vector_index	vectorSearch	description_vector	ACTIVE View status details	Primary Node: 55.53KB	Primary Node: 22 (100%) indexed of 22 total	<button>QUERY</button> <button>...</button>

That is all for this step. You successfully created a search index. The next step is to create the service to use this index.

Task 6: Create service endpoint

Your task is to create an endpoint similar to what you did in the last lab [News Finder Service with MongoDB Atlas](#). The endpoint will take a sentence as input and return the books with the plots closest to that sentence. You can create a new file called `server.js` and add this code.

```
touch server.js
```

However, there are some differences in the agg function. You will return `book_name` and `summaries` with the results.

```
const agg = [
  {
    '$vectorSearch': {
      'index': 'vector_index',
      'path': 'description_vector',
      'queryVector': query,
      'numCandidates': 5,
      'limit': 5
    }
  },
  {
    '$project': {
      '_id': 0,
      'book_name': 1,
      'summaries': 1,
      'score': {
        '$meta': 'vectorSearchScore'
      }
    }
  },
  {
    '$match': {
      'score': { '$gt': 0.1 }
    }
  }
];
```

Here is the complete file if you get stuck.

1. [Click for a hint](#)

► [Click here for the answer](#)

Task 7: Run the express application

You will run the file with `node server.js`.

Let's see this in action using the `curl` command that looks for plots for a story with a story with robots. The expectation is to get results based on computers or technology.

```
curl -X POST http://localhost:3000/search \
-H "Content-Type: application/json" \
-d '{"text": "a story about rotbots"}' | jq .
```

Result:

```
{
  "book_name": "1984",
  "summaries": " is the story of a man questioning the system that keeps his futuristic but dystopian society afloat and the chaos that quickly ens",
  "score": 0.743269681930542
},
{
```

```

    "book_name": "1984",
    "summaries": " is the story of a man questioning the system that keeps his futuristic but dystopian society afloat and the chaos that quickly ens
    "score": 0.743269681930542
  },
  {
    "book_name": "The Code Breaker",
    "summaries": " details the life of Nobel Prize winner Jennifer Doudna, who embarked on – and successfully completed – a journey to invent a tool
    "score": 0.7224152088165283
  },
  {
    "book_name": "Brave New World",
    "summaries": " presents a futuristic society engineered perfectly around capitalism and scientific efficiency, in which everyone is happy, confor
    "score": 0.7051411867141724
  },
  {
    "book_name": "Brave New World",
    "summaries": " presents a futuristic society engineered perfectly around capitalism and scientific efficiency, in which everyone is happy, confor
    "score": 0.7051411867141724
  }
]

```

Congratulations! You successfully used the MongoDB Atlas database, created vector embeddings for book plot data, created an index on this vector, and finally created an endpoint that can be used to retrieve similar books!

Authors

UL